HDOC Day-50

Finding the Majority Element in an Array Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, user-defined functions, pointers, and loops to count occurrences of distinct elements and identify a majority element appearing strictly more than size / 2 times. Display -1 when no majority element exists.

Algorithm

1. Start the program and read the array size.
2. Validate that size is positive; otherwise display an error.
3. Declare the variable-length array nums[size] and read its elements.
4. Pass nums and size to majority_element() using the array address.
5. For each distinct element, count its occurrences using nested loops and display the count.
6. Search for an element whose count is strictly greater than size / 2.
7. Display that majority element and return from the function, or display -1 if none exists.
8. Stop the program.

C Program

```
#include <stdio.h>

void majority_element(int *nums, int size);

int main()
{
    int size, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int nums[size];
    printf("Enter %d array elements: ", size);

    for (i = 0; i < size; i++)
        scanf("%d", &nums[i]);

    majority_element(nums, size);
    return 0;
}

/* Function to find the majority element */
void majority_element(int *nums, int size)
{
    int i, j, count, alreadyCounted;

    printf("\nOccurrences of distinct elements:\n");

    for (i = 0; i < size; i++)
    {
        alreadyCounted = 0;
        for (j = 0; j < i; j++)
        {
            if (*(nums + i) == *(nums + j))
            {
                alreadyCounted = 1;
                break;
            }
        }
        if (alreadyCounted)
            continue;

        count = 0;
        for (j = 0; j < size; j++)
        {
            if (*(nums + i) == *(nums + j))
                count++;
        }
        printf("Element %d occurs %d times.\n",
               *(nums + i), count);
    }

    for (i = 0; i < size; i++)
    {
        count = 0;
        for (j = 0; j < size; j++)
        {
            if (*(nums + i) == *(nums + j))
```

```
                count++;
        }
        if (count > size / 2)
        {
            printf("\nMajority Element: %d\n", *(nums + i));
            return;
        }
    }
    printf("\nMajority Element: -1\n");
}
```

Explanation

The call majority_element(nums, size) passes the address of the first array element. Inside the function, *(nums + i) accesses nums[i]. An element qualifies only when its frequency is greater than size / 2; merely having the highest frequency is insufficient.

Example: In the array 2 2 1 2 3 2 2, the size is 7 and the value 2 occurs 5 times. Since 5 > 7 / 2, the majority element is 2.

Test Case Table

Case	Input Array	Expected Result
1	2 2 1 2 3 2 2	Majority Element = 2
2	3 3 4 2 3 3 5	Majority Element = 3
3	1 2 3 4 5	-1
4	4 4 4 4 4	Majority Element = 4
5	1 1 2 2	-1
6	7	Majority Element = 7
7	5 5 5 2 2 5	Majority Element = 5
8	-1 -1 -1 2 3	Majority Element = -1
9	size = 0	Invalid array size

Note: For Test Case 8, -1 is itself the majority element. The assignment also uses -1 to indicate that no majority element exists.

Sample Outputs

Majority Element Exists

```
Enter the size of array: 7
Enter 7 array elements: 2 2 1 2 3 2 2

Occurrences of distinct elements:
Element 2 occurs 5 times.
Element 1 occurs 1 times.
Element 3 occurs 1 times.

Majority Element: 2
```

No Majority Element

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5

Occurrences of distinct elements:
Element 1 occurs 1 times.
Element 2 occurs 1 times.
Element 3 occurs 1 times.
Element 4 occurs 1 times.
Element 5 occurs 1 times.

Majority Element: -1
```

All Elements Equal

```
Enter the size of array: 5
Enter 5 array elements: 4 4 4 4 4

Occurrences of distinct elements:
Element 4 occurs 5 times.

Majority Element: 4
```

Invalid Array Size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program was designed using variable-length arrays, pointers, user-defined functions, nested loops, and conditional statements. The majority_element() function counts occurrences of distinct elements, displays a majority element when its frequency exceeds size / 2, and otherwise displays -1.